4. Vector2 y Vector3

Los tipos **Vector2** y **Vector3** son estructuras fundamentales en Unity para representar posiciones, direcciones, desplazamientos y cálculos matemáticos dentro del mundo del juego. Son herramientas indispensables en movimiento, física, animaciones, detección de colisiones, cálculos de distancias y más.

4.1. Vector2

Un **Vector2** contiene dos componentes: **x** e **y**.
Se utiliza principalmente en entornos 2D para representar:

- Posiciones en un plano 2D
- Direcciones y desplazamientos
- Inputs del jugador (palanca, teclado, joystick)
- Velocidades 2D

Ejemplo real del script **Jugador**, donde se lee la dirección del input:

```
Vector2 direccion = mover.action.ReadValue<Vector2>();

transform.position += velocidad * (Vector3)direccion *
Time.deltaTime;
```

Aquí, el Vector2 obtenido se convierte a Vector3 para poder modificar la posición del Transform.

4.2. Vector3

Un **Vector3** contiene tres componentes: **x**, **y**, **z**.
Es el vector estándar para trabajar con:

- Posiciones 3D
- Transform de cualquier GameObject
- Rotaciones (cuando son expresadas en eulerAngles)
- Movimientos en 2D (aunque el valor Z suele ser 0)
- Cálculos de orientación

Ejemplo tomado del script **Enemigo**:

```
transform.position += new Vector3(velocidad, 0, 0) *
Time.deltaTime;
```

Y para instanciar objetos desplazados respecto a la posición actual:

```
Instantiate(prefabBalaE, transform.position + Vector3.down,
Quaternion.Euler(0, 0, -90));
```

El uso de **Vector3.down** da acceso directo a un vector predefinido.

4.3. Conversiones entre Vector2 y Vector3

En proyectos 2D es habitual pasar de Vector2 a Vector3.
Unity permite conversiones explícitas:

```
(Vector3)direccion   // De Vector2 → Vector3
```

Útil cuando un input o cálculo 2D se aplica al Transform, que requiere Vector3.

4.4. Propiedades y métodos útiles

Unity ofrece múltiples utilidades para trabajar con vectores:

Vectores predefinidos

- Vector2.zero, Vector3.zero
- Vector2.one, Vector3.one
- Vector3.up, Vector3.down, Vector3.left, Vector3.right

Ejemplo tomado del script **Jugador**, para disparos en varias direcciones:

```
CrearBala(new Vector3(0, 0.5f, 0), 90);
```

Aquí se define una dirección desplazada mediante un Vector3 literal.

Normalización

Convierte el vector en uno de magnitud 1:

```
Vector2 dir = direccion.normalized;
```

Muy útil para controlar direcciones independientemente de la distancia.

Magnitud y distancia

Propiedades frecuentes en cálculos de movimiento:

```
float distancia = (posA - posB).magnitude;
```

Lerp y MoveTowards

Herramientas valiosas para interpolación:

```
Vector3.Lerp(a, b, t);
```

```
Vector3.MoveTowards(actual, destino, velocidad *
Time.deltaTime);
```

Son esenciales en movimiento de enemigos, cámaras o efectos visuales.

4.5. Vectores y Time.deltaTime

En la mayoría de scripts del proyecto, el movimiento se calcula multiplicando un vector por **Time.deltaTime**, lo que garantiza un movimiento fluido y consistente:

```
transform.position += velocidad * (Vector3)direccion *
Time.deltaTime;
```

Sin deltaTime, el movimiento dependería de los FPS, produciendo inconsistencias.

4.6. Vectores en disparos y orientaciones

En el script **Bala**, los vectores se usan para avanzar en la dirección del objeto:

```
transform.position += velocidad * transform.right *
Time.deltaTime;
```

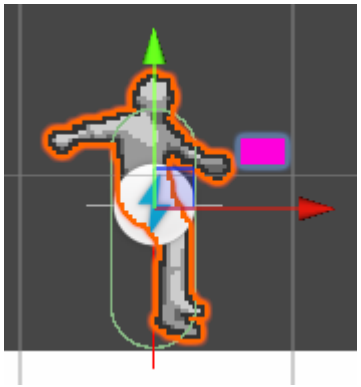
transform.right es un Vector3 que indica hacia qué lado apunta el objeto, lo cual permite disparos orientados sin necesitar cálculos adicionales.

4.7. Vectores en sensores y físicas

En el script **Humano**, un Vector2 se utiliza para lanzar un rayo hacia abajo y detectar el suelo:

```
RaycastHit2D hit = Physics2D.Raycast(transform.position,
Vector2.down, distanciaSensorSuelo, mascaraSuelo);
```

Aquí se combinan un punto de origen (Vector3 implícito), una dirección 2D y una distancia.



4.8. Vectores como herramienta transversal

Los vectores permiten:

- Movimiento continuo
- Detectar direcciones
- Definir offsets para proyectiles
- Crear desplazamientos relativos
- Realizar cálculos matemáticos
- Establecer puntos de aparición
- Animar interpolaciones

Son uno de los recursos más utilizados en cualquier proyecto Unity, desde mecánicas simples a sistemas complejos de IA o física.